
Final 1Jul2025

1) Explique el concepto de Split Brain. ¿Puede ocurrir utilizando el framework Hazelcast? Si es así, ¿cómo?

Respuesta: Split Brain es un problema que se da en cualquier red de varios sistemas distribuidos, y surge cuando, por motivos de comunicación, el cluster se divide en 2 partes autónomas, funcionando de manera independiente. Esto trae otro problema a la hora de reconectar los nodos, ya que se deben mergear los datos. Esto puede suceder en Hazelcast, por ejemplo cuando se cae el nodo maestro, por lo que provee un mecanismo para realizar la recuperación.

2) Indicar qué significa que un sistema de store cumpla con el modelo de consistencia eventual.

Respuesta: Un sistema de store cumple con el modelo de consistencia eventual si pasado un tiempo de una escritura realizada sin otras en el medio, todas las lecturas reflejan este mismo valor.

3)Cuál es la utilidad del Batch Loader en GraphQL.

Respuesta: La utilidad del Batch Loader en GraphQL es mitigar el problema de las N+1 queries, que se produce cuando se debe se realiza una primera query para obtener N elementos, y luego N queries para cada uno de ellos. Lo que hace el Batch Loader es acumular los pedidos al campo y en vez de hacer n llamados con 1 elemento, hace 1 llamado con los n elementos en un array.

4) Indicar 3 características que deben cumplir las APIs REST

Respuesta: Tres características con las que cumplen las API REST son:

- Stateless: El estado/contexto de la petición es mantenida por el cliente.
- Cacheable: Todos los recursos deben ser cacheables, mientras y por el tiempo que tenga sentido.
- Arquitectura cliente-servidor (Client-Server)

5) Qué significa que un sistema distribuido sea elástico.

Respuesta: Un sistema distribuido es elástico si tiene la capacidad de modificar la cantidad de nodos, y que los nuevos puedan comenzar a procesar sin tener que desconectar todo el servicio.

6) Supongamos un dataset de árboles de una ciudad que modela sus datos de la siguiente forma:

- id: identificador autogenerado
- barrio: Nombre del barrio donde se encuentra el árbol.
- calle: Nombre de la calle donde se encuentra el árbol
- especie: Especie del árbol

Suponiendo que se quiere utilizar un proceso mapreduce (que puede ser una cadena de procesos). En este proceso se recibe en el mapper inicial el id del árbol como clave y el resto de los datos del registro como valor. Se pide sin codificar, indique en palabras. Para cada job en la cadena

- ¿Cuáles serían el comportamiento del Mapper?
- ¿Cuál es la clave y el valor que emite mapper?
- ¿Cuál sería el comportamiento del Reducer?
- ¿Cuál es la clave y el valor que emite Reducer?
- Indicar si hay que hacer un procesamiento posterior de los resultados

Para el caso en que se quiera resolver la siguiente consulta: El top ten de barrios con mayor cantidad de especies distintas ordenado descendente por cantidad.

Respuesta: En este caso, el mapper se encarga de emitir, por cada árbol, el barrio como clave y la especie como valor. Luego, el reducer arma un set para cada barrio con las especies, y devuelve el tamaño del set. Por último, se implementa un Collator que postprocesa los datos ordenando descendente por la cantidad, y se queda solamente con los 10 primeros registros.

7) Supongamos un sitio de reviews de películas que modela sus datos:

- id: identificador autogenerado
- título: nombre de las películas.
- director: Lista de los directores (cada uno un string de nombres y apellidos)
- Protagonistas: Lista de protagonistas (cada uno un string de nombres y apellidos).
- Género: Texto que indica el género principal de la película.
- Clasificación: valor que indica si la película es apta para todo público o tiene alguna restricción.

- Fecha de estreno.
- Taquilla: Recaudación total de la película.
- Reviews: Lista de reviews donde cada review tiene:
 - username: identificador del usuario.
 - rating: calificación numérica 1 a 10.
 - comentario: texto libre para indicar razones de la calificación.
 - likes: cantidad de likes dados por otros usuarios al review.

Tener en cuenta que el nombre de una película puede aparecer muchas veces ya sea por remakes o por películas que simplemente se llaman igual. Suponiendo que se quiere utilizar un proceso map-reduce (que puede ser una cadena de procesos). En este proceso se recibe en el mapper inicial el id de la película como clave y el resto de los datos del registro como valor. Se pide sin codificar, indique en palabras. Para cada job en la cadena.

1. ¿Cuáles serían el comportamiento del Mapper?
2. ¿Cuál es la clave y el valor que emite mapper?
3. ¿Cuál sería el comportamiento del Reducer?
4. ¿Cuál es la clave y el valor que emite reducer?
5. Indicar si hay que hacer un procesamiento posterior de los resultados

Para cada década cuál fue el director más taquillero de la misma (contando todas sus películas de la misma).

Respuesta: El mapper en este caso se encarga de, para cada película, emitir la década de la película en base a la fecha de estreno, junto con el director y la taquilla como valor. Luego, el reducer contiene un mapa con el director como clave y la taquilla como valor. Si ya se registró el director, se suma el nuevo valor de taquilla. Finalmente, retorna la clave de mayor suma total en taquilla. No se requiere postprocesamiento.

8) ¿Cómo se produce un LiveLock?

Respuesta: El LiveLock es un fenómeno que se produce cuando se intenta solucionar un deadlock (dos procesos intentan acceder a dos locks uno luego del otro y nunca pueden tomar el segundo) mediante un tiempo de espera y reintento, pero este ciclo se repite infinitamente.

9) ¿Cuál es el objetivo buscado al utilizar ExecutorService?

Respuesta: La interfaz de ExecutorService se utiliza para centralizar la creación y el manejo del ciclo de vida de Thread, y que los mismos se wrapeen con un Future para que se pueda utilizar esa semántica para acceder al resultado.

10) ¿Las bases de datos Columnares por su manera de guardar son muy eficientes para qué tipo de consultas?

Respuesta: Las bases de datos columnares son muy eficientes para consultas de agregación y analíticas, ya que guardan toda la información por columnas.

Final 1Dic2024

1)Cuál es la utilidad del Batch Loader en GraphQL.

Respuesta: La utilidad del Batch Loader en GraphQL es mitigar el problema de las N+1 queries, que se produce cuando se debe se realiza una primera query para obtener N elementos, y luego N queries para cada uno de ellos. Lo que hace el Batch Loader es acumular los pedidos al campo y en vez de hacer n llamados con 1 elemento, hace 1 llamado con los n elementos en un array.

2) Qué significa cuando se indica que una API es basada en eventos e indicar algún ejemplo de este tipo de API.

Respuesta: El modelo basado en eventos consiste en que el cliente se registra para la recepción de cierto tipo de eventos, como por ejemplo la creación de un nuevo usuario, para que el servidor se los envíe una vez que suceden. Un ejemplo de esto son las APIs pub/sub.

3) Supongamos un dataset de molinetes de estaciones de subte de una ciudad que modela sus datos de la siguiente forma:

- id: identificador autogenerado
- estación: Nombre de la estación donde se encuentra el molinete.
- línea: Nombre de la línea de subte a la cual pertenece la estación donde se encuentra el molinete
- fechaHora: Fecha y hora de la medición del molinete
- pasajeros: Cantidad de pasajeros que pasaron por el molinete en la fecha indicada
- donde existen hasta 24 registros por día para cada molinete (uno por hora).

Suponiendo que se quiere utilizar un proceso mapreduce (que puede ser una cadena de procesos). En este proceso se recibe en el mapper inicial el id de la medición como clave y el resto de los datos del registro como valor. Se pide sin codificar, indique en palabras para cada job en la cadena:

- ¿Cuáles serían el comportamiento del Mapper?
- ¿Cuál es la clave y el valor que emite mapper?
- ¿Cuál sería el comportamiento del Reducer?
- ¿Cuál es la clave y el valor que emite Reducer?
- Indicar si hay que hacer un procesamiento posterior de los resultados

Para el caso en que se quiera resolver la siguiente consulta: Estación con más pasajeros de cada línea de subte para el año 2023, indicando el nombre de la estación, el nombre de la línea y la cantidad total de pasajeros que pasaron por todos los molinetes de esa estación para el año 2023. El orden es alfabético por nombre de la línea.

Respuesta: En este caso, para cada molinete, se devuelve como clave la línea de subte, y como valor la estación y el número de pasajeros, siempre y cuando el año sea 2023. Luego, el reducer se encarga de armar un mapa con las estaciones como clave y va sumando los pasajeros, para retornar la clave con la mayor suma total. Por último, se ordena alfabéticamente por nombre de la línea.

4) ¿Por qué son preferibles los objetos inmutables en ambientes concurrentes?

Respuesta: Los objetos inmutables son preferibles porque, al no poder ser modificados, solucionan problemas de concurrencia y consistencia en el uso de Threads.

5) ¿Cómo se produce un LiveLock?

Respuesta: El LiveLock es un fenómeno que se produce cuando se intenta solucionar un deadlock mediante un tiempo de espera y reintento, pero este ciclo se repite infinitamente.

6) Indicar si es correcto que la variable counter tiene su lectura y escritura debidamente protegidas en el siguiente código (donde se ve toda la interacción de la clase con dicha variable).

```
public class Servlet {
    private Object lockCounterLock;
    private int counter;

    public void responseRequest(Request request) {
        ...
        synchronized (lockCounterLock) {
            ...
            counter++;
        }
        ...
    }

    public synchronized int getCount() {
        return counter;
    }
}
```

Respuesta: No, no es correcto, la variable counter no tiene su lectura y escritura debidamente protegidas, ya que en la responseRequest se la está bloqueando con el objeto lockCounterLock, pero en el método getCount el synchronized utiliza el objeto this para el bloqueo, por lo que pueden ocurrir race conditions.

7) ¿Qué es lo que se indica al decir que una base de datos provee consistencia eventual?

Respuesta: Al decir que una base de datos provee consistencia eventual se indica que al realizar una escritura por parte del cliente, todas las lecturas serán consistentes luego de un cierto tiempo sin nuevas escrituras.

8) Explique el concepto de Split Brain. ¿Puede ocurrir utilizando el framework Hazelcast? Si es así, ¿cómo?

Respuesta: Split Brain es un problema que se da en cualquier red de varios sistemas distribuidos, y surge cuando, por motivos de comunicación, el cluster se divide en 2 partes autónomas, funcionando de manera independiente. Esto trae otro problema a la hora de reconectar los nodos, ya que se deben mergear los datos. Esto puede suceder en Hazelcast por ejemplo con la caída del nodo maestro, por lo que provee un mecanismo para realizar la recuperación.

9) A la hora de tener trabajo distribuido indicar que son y para que se usan:

- particionado
- replicación

Respuesta: El particionado consiste en distribuir la información en distintos nodos, lo que permite tener escrituras y lecturas más eficientes. Pero al tener una sola copia en cada nodo puede traer problemas si, por ejemplo, uno se cae. Entonces se realiza una replicación de los datos, es decir que se guardan repetidos en distintos nodos para contemplar esta situación problemática.

10) En Hazelcast, por qué es necesario implementar el método reset() al escribir un Combiner.

Respuesta: Es necesario implementarlo porque el Combiner no espera a que lleguen todos los datos, sino que trabaja con chunks que emite con el resultado parcial. Al acabarse el mismo, se debe utilizar el método `reset()` para reiniciar el contador y recibir nuevos datos para armar el nuevo resultado parcial, para así utilizar una única instancia.

Final 2Dic2023

1) Indicar 3 características que deben cumplir las APIs REST.

Respuesta: Tres características con las que deben cumplir las API REST son:

- 1) Arquitectura Cliente-Servidor
- 2) Stateless: el cliente mantiene el estado de una petición, no el servidor.
- 3) Cacheable: los recursos debe poder ser cacheables por un tiempo acorde.

2) Una instancia de la siguiente clase se quiere usar para compartir entre threads, los potenciales valores de estados de una aplicación. Indicar, si existen, cuales son los riesgos de utilizar dicha instancia y las posibles soluciones para mitigar los problemas indicados.

```
public class States {  
    ...  
    private final String[] states = new String[] { "UNK", "NEW", "RUNNING", "DONE" };  
    public String[] getStates() {  
        return states;  
    }  
}
```

Respuesta: Los riesgos de usar una instancia de esta clase están en que el final de states, únicamente garantiza que la referencia no cambiará, pero los elementos sí pueden ser modificados. Para arreglar este problema, podría devolverse una copia del array.

3) Que significa que un sistema distribuido sea elástico.

Respuesta: Que un sistema distribuido sea elástico significa que su número de nodos puede ser modificado y los nuevos pueden comenzar a operar directamente sin tener que desconectar todo el clúster.

4) En hazelcast por defecto ¿Cómo se selecciona al nodo dueño de una partición?

Respuesta: En Hazelcast, el dueño de una partición se elige utilizando el algoritmo de consistent hashing. El nodo que tiene la mayor antigüedad en el clúster es seleccionado como el Coordinador y mantiene una tabla de particiones que indica qué nodo es el dueño (owner) y cuál es el backup de cada una de las 271 particiones.

Para decidir a qué partición pertenece un objeto, se calcula el número de partición aplicando un hash a la clave del objeto y usando el módulo 271.

5) Explique el concepto de Split Brain. ¿Puede ocurrir utilizando el framework Hazelcast? Si es así, ¿cómo?

Respuesta: Split Brain es un problema que se da en cualquier red de varios sistemas distribuidos, y surge cuando, por motivos de comunicación, el cluster se divide en 2 partes autónomas, funcionando de manera independiente. Esto trae otro problema a la hora de reconectar los nodos, ya que se deben mergear los datos. Esto puede suceder en Hazelcast por ejemplo con la caída del nodo maestro, por lo que provee un mecanismo para realizar la recuperación.

6) Explicar la estrategia de Cassandra realiza la escritura y replicación de un dato.

Respuesta: En Cassandra, cualquier nodo puede recibir la escritura del cliente. Lo primero que hace, es verificar a qué nodo corresponde la partición. Luego, lo envía a dicho nodo y a los siguientes nodos del anillo determinados según el replication factor definido en la base de datos. Una vez que recibe el ACK de la cantidad de nodos definida en el consistency level, confirma la escritura al cliente.

7) Qué es el factor de replicación y cómo puede afectar a la latencia y consistencia a la hora de realizar escrituras en una base de datos NoSQL.

Respuesta: El factor de replicación determina el número de copias de un dato que deben ser guardadas en la base de datos. Es decir, con un RF de 3, el dato vivirá en 3 nodos distintos. Si se aumenta el factor de replicación, se obtiene una consistencia mayor para la realización de escrituras, pero al mismo tiempo se aumenta la latencia, ya que más nodos deben confirmar la operación antes de avisar al cliente. Lo contrario sucede en ambos casos si se toma un RF bajo.

8) Indicar si es correcto que en el siguiente código:

```

public class Servlet {
    private Object lockCounterLock;
    private int counter;
    public void responseRequest(Request request) {
        ...
        synchronized (lockCounterLock) {
            ...
            counter++;
        }
        ...
    }
    public synchronized int getCount(){
        return counter;
    }
}

```

La instrucción

```

    synchronized (lockCounterLock) {
        ...
    }

```

indica que ningún otro código de la clase puede modificar la variable lockCounterLock.

Respuesta: Es incorrecto, porque el método getCount() puede acceder a la variable count, ya que está bloqueándose con la instancia de this, dado que se declaró synchronized en el método. En cambio, el método responseRequest se bloquea con una instancia del Object lockCounterLock. Es verdad que getCount no lo modifica, pero podría hacerlo tranquilamente porque no está bien protegido.

9) Supongamos un dataset de movimientos de un aeropuerto (despegues y aterrizajes) que modela sus datos de la siguiente forma:

- id: identificador autogenerado
- tipo: Despegue o Aterrizaje.
- origen: Código del aeropuerto del cual despegó el vuelo.
- destino: Código del aeropuerto en el cual aterrizó el vuelo

Suponiendo que se quiere utilizar un proceso map reduce (que puede ser una cadena de procesos). En este proceso se recibe en el mapper inicial el id del movimiento como clave y el resto de los datos del registro como valor. Se pide sin codificar, indique en palabras para cada job en la cadena

- ¿Cuáles serían el comportamiento del Mapper?
- ¿Cuál es la clave y el valor que emite Mapper?
- ¿Cuál sería el comportamiento del Reducer?
- ¿Cuál es la clave y el valor que emite Reducer?
- Indicar si hay que hacer un procesamiento posterior de los resultados

Para el caso en que se quiera resolver la siguiente consulta: Top 10 aeropuertos destino con mayor cantidad de despegues que tienen como origen al aeropuerto EZE, ordenado descendente por cantidad de despegues.

Respuesta: El mapper recibe los movimientos y se encarga de emitir como clave el id de destino y como valor un 1, omitiendo los casos donde el origen no sea EZE. Luego, el reducer produce la sumatoria de todos los viajes para cada id de destino y la retorna. Finalmente, se requiere de postproceso para ordenar y obtener únicamente los primeros 10, para lo que se puede usar un Collator.

10) Supongamos un dataset de molinetes de estaciones de subte de una ciudad que modela sus datos de la siguiente forma:

- id: identificador autogenerado
- estación: Nombre de la estación donde se encuentra el molinete.
- línea: Nombre de la línea de subte a la cual pertenece la estación donde se encuentra el molinete
- fechaHora: Fecha y hora de la medición del molinete
- pasajeros: Cantidad de pasajeros que pasaron por el molinete en la fecha indicada donde existen hasta 24 registros por día para cada molinete (uno por hora).

Suponiendo que se quiere utilizar un proceso map reduce (que puede ser una cadena de procesos). En este proceso se recibe en el mapper inicial el id de la medición como clave y el resto de los datos del registro como valor. Se pide sin codificar, indique en palabras para cada job en la cadena

- ¿Cuáles serían el comportamiento del Mapper?
- ¿Cuál es la clave y el valor que emite mapper?
- ¿Cuál sería el comportamiento del Reducer?
- ¿Cuál es la clave y el valor que emite Reducer?
- Indicar si hay que hacer un procesamiento posterior de los resultados

Para el caso en que se quiera resolver la siguiente consulta: Estación con más pasajeros de cada línea de subte para el año 2021, indicando el nombre de la estación, el nombre de la línea y la cantidad total de pasajeros que pasaron por todos los molinetes de esa estación para el año 2021. El orden es alfabético por nombre de la línea.

Respuesta: En este caso, para cada molinete, se devuelve como clave la línea de subte, y como valor la estación y el número de pasajeros, siempre y cuando el año

sea 2021. Luego, el reducer se encarga de armar un mapa con las estaciones como clave y va sumando los pasajeros, para retornar la clave con la mayor suma total. Por último, se ordena alfabéticamente por nombre de la línea.

Final 1Dic2023

1) ¿Qué es lo que se indica al decir que una base de datos provee consistencia eventual?

Respuesta: Un sistema de store cumple con el modelo de consistencia eventual si pasado un tiempo de una escritura realizada sin otras en el medio, todas las lecturas reflejan este mismo valor.

2) ¿Las bases de datos Columnares por su manera de guardar son muy eficientes para qué tipo de consultas?

Respuesta: Al guardar toda la información por columnas, las bases de datos columnares son muy eficientes para consultas que involucren agregaciones y analítica sobre los datos.

3) Para el siguiente código indique si el mismo es apto para utilizar en un ambiente concurrente. Justifique en ambos casos y en caso de no serlo proponga una solución.

```
public class Friend {  
    private final String name;  
    public Friend(String name) {  
        this.name = name;  
    }  
    public String getName() {  
        return this.name;  
    }  
    public synchronized void bow(Friend bower) {  
        System.out.format("%s: %s"+" has bowed to me!\n",this.name, bower.getName());  
        bower.bowBack(this);  
    }  
    public synchronized void bowBack(Friend bower) {  
        System.out.format("%s: %s"+" has bowed back to me!\n",this.name, bower.getName());  
    }  
}
```

Respuesta: No, el caso no es apto para utilizar en un ambiente concurrente porque supongamos que dos amigos quieren hacer la reverencia, puede generarse un deadlock donde cada uno está esperando que el otro “suelte” el lock. [FALTA LA SOLUCIÓN]

4) Supongamos un dataset de árboles de una ciudad que modela sus datos de la siguiente forma:

- id: identificador autogenerado
- barrio: Nombre del barrio donde se encuentra el árbol.
- calle: Nombre de la calle donde se encuentra el árbol
- especie: Especie del árbol

Suponiendo que se quiere utilizar un proceso mapreduce (que puede ser una cadena de procesos). En este proceso se recibe en el mapper inicial el id del árbol como clave y el resto de los datos del registro como valor. Se pide sin codificar, indique en palabras. Para cada job en la cadena

- ¿Cuáles serían el comportamiento del Mapper?
- ¿Cuál es la clave y el valor que emite mapper?
- ¿Cuál sería el comportamiento del Reducer?
- ¿Cuál es la clave y el valor que emite Reducer?
- Indicar si hay que hacer un procesamiento posterior de los resultados

Para el caso en que se quiera resolver la siguiente consulta: El top ten de barrios con mayor cantidad de especies distintas ordenado descendente por cantidad.

Respuesta: En este caso, el mapper se encarga de emitir, por cada árbol, el barrio como clave y la especie como valor. Luego, el reducer arma un set para cada barrio con las especies, y devuelve el tamaño del set. Por último, se implementa un Collator que postprocesa los datos ordenando descendente por la cantidad, y se queda solamente con los 10 primeros registros.

5) Explique el concepto de Split Brain. ¿Puede ocurrir utilizando el framework Hazelcast? Si es así, ¿cómo?

Respuesta: Split Brain es un problema que se genera en cualquier sistema distribuido, donde por un problema de comunicación, el mismo queda dividido en dos partes autónomas que continúan operando. Esto trae otro problema a la hora de reconectar los nodos, ya que se deben mergear los datos. Esto puede suceder en Hazelcast por ejemplo cuando se cae el nodo maestro, por lo que provee un mecanismo para la recuperación.

6) Explicar cuál es la función del Combiner en el paradigma de programación distribuido MapReduce.

Respuesta: En el paradigma de programación distribuida MapReduce, un Combiner se utiliza para realizar un procesamiento en el mismo nodo del mapper, luego de la ejecución de este y previo a la etapa de sort del framework. Su objetivo principal es optimizar la eficiencia del procesamiento al resumir o acumular localmente los datos generados por el Mapper antes de que sean transmitidos a través de la red.

7) Supongamos un sitio de reviews de películas que modela sus datos:

- id: identificador autogenerado
- título: nombre de las películas.
- director: Lista de los directores (cada uno un string de nombres y apellidos)
- Protagonistas: Lista de protagonistas (cada uno un string de nombres y apellidos).
- Género: Texto que indica el género principal de la película.
- Clasificación: valor que indica si la película es apta para todo público o tiene alguna restricción.
- Fecha de estreno.
- Taquilla: Recaudación total de la película.
- Reviews: Lista de reviews donde cada review tiene:
 - username: identificador del usuario.
 - rating: calificación numérica 1 a 10.
 - comentario: texto libre para indicar razones de la calificación.
 - likes: cantidad de likes dados por otros usuarios al review.

Tener en cuenta que el nombre de una película puede aparecer muchas veces ya sea por remakes o por películas que simplemente se llaman igual. Suponiendo que se quiere utilizar un proceso map reduce (que puede ser una cadena de procesos). En este proceso se recibe en el mapper inicial el id de la película como clave y el resto de los datos del registro como valor. Se pide sin codificar, indique en palabras para cada job en la cadena:

- ¿Cuáles serían el comportamiento del Mapper?
- ¿Cuál es la clave y el valor que emite mapper?
- ¿Cuál sería el comportamiento del Reducer?
- ¿Cuál es la clave y el valor que emite reducer?
- Indicar si hay que hacer un procesamiento posterior de los resultados.

Para: Indicar el reviewer ochentoso más popular donde popularidad es la suma de los likes de los ratings que publicó y "ochentoso" porque solo se toman las reviews para películas de los 80s.

Respuesta: En este caso, el mapper va a iterar por sobre la lista de reviews para cada película y emitir por cada una el username como clave y los likes como valor, siempre y cuando la película sea de los 80s. Luego, el reducer suma todos los likes que recibió cada reviewer, y retorna la suma. El más popular se obtiene mediante postproceso.

8) Indicar cuál es la función del Stub en un ambiente cliente servidor usando grpc.

Respuesta: Un stub en un ambiente cliente servidor que utiliza gRPC es un proxy de objetos remotos en el cliente. Su rol principal es garantizar que las llamadas al servicio sean lo más "transparentes" posibles, de modo que no se distingan de las llamadas a métodos locales. Se encarga de recibir las invocaciones del cliente y pasarlas por la red (el método invocado y los parámetros), y esperar los resultados para leerlos y transmitirlos al cliente.

9) Indicar 3 características que deben cumplir las APIs REST.

Respuesta: Tres características con las que deben cumplir las API REST son:

1. Arquitectura Cliente-Servidor
2. Stateless: el cliente mantiene el estado de una petición, no el servidor.
3. Cacheable: los recursos debe poder ser cacheables por un tiempo acorde.

10)Cuál es la utilidad del Batch Loader en GraphQL.

Respuesta: La utilidad del Batch Loader en GraphQL es mitigar el problema de las N+1 queries, que se produce cuando se debe se realiza una primera query para obtener N elementos, y luego N queries para cada uno de ellos. Lo que hace el Batch Loader es acumular los pedidos al campo y en vez de hacer n llamados con 1 elemento, hace 1 llamado con los n elementos en un array.